FULL STACK DEVELOPMENT WITH MERN

Project Documentation Report

OrderOnTheGo

Your On-Demand Food Ordering Solution

Submitted By:

Dara Chilakamma

Ramadi Pavani Durga

Yalla Sri sai Pranay

Dhanush Srikakulapu

Submitted To:

Anji Babu

SmartBridge | SmartInternz | APSCHE

On:

Date: February 2026

Table of Contents

1. Introduction 3
2. Project Overview 4
3. Architecture 6
4. Setup Instructions 8
5. Folder Structure 10
6. Running the Application 12
7. API Documentation 13
8. Authentication 15
9. User Interface 17
10. Testing 19
11. Screenshots and Demo 21
12. Known Issues 26
13. Future Enhancements 27
14. Conclusion 29
15. References 30
Acknowledgments 30

1. Introduction

1.1 Project Title

OrderOnTheGo (SB Foods) — Your On-Demand Food Ordering Solution

1.2 Team Members

The following individuals contributed to the development and implementation of the OrderOnTheGo project:

Dara Chilakamma
Ramadi Pavani Durga
Yalla Sri sai Pranay
Dhanush Srikakulapu

1.3 Project Duration

The project was developed as part of the Full Stack Development with MERN curriculum under the SmartBridge internship program during 15th december,2025 to 28th feburary,2026 phase.

1.4 Technology Stack

- Frontend: React.js, HTML5, CSS3, Bootstrap
- Backend: Node.js, Express.js
- Database: MongoDB (via Mongoose ODM)
- Version Control: Git / GitHub
- Development Environment: Visual Studio Code

2. Project Overview

2.1 Purpose

OrderOnTheGo is a cutting-edge, full-stack digital food ordering platform built to revolutionize the way users discover, browse, and order food online. The application addresses the growing demand for convenient, contactless food delivery services by providing an intuitive web-based interface that connects customers with restaurants in real time.

The platform is designed to eliminate the friction associated with traditional food ordering — long queues, phone calls, unclear menus, and uncertain delivery timelines — and replace them with a seamless, transparent, and efficient digital experience accessible from any device.

2.2 Project Goals

The key objectives driving the development of OrderOnTheGo are as follows:

- Deliver a responsive, mobile-friendly web application for on-demand food ordering
- Implement a secure multi-role authentication system for Users, Restaurant Owners, and Administrators
- Provide a comprehensive product catalog with filtering and search capabilities
- Enable a smooth cart-to-checkout order flow with real-time confirmation
- Build an Admin Dashboard for platform-level management of users, orders, and restaurants
- Develop a Restaurant Dashboard enabling restaurant owners to manage menus and track orders
- Ensure scalability and maintainability through a modular MERN stack architecture

2.3 Key Features and Functionalities

2.3.1 User Features

- User Registration and Login: Secure account creation with password-based authentication
- Restaurant & Menu Browsing: Explore restaurants and view detailed food item listings with descriptions, pricing, and reviews
- Shopping Cart: Add, update, and remove food items before placing an order
- Order Placement: Provide delivery address and payment method, then place order with instant confirmation
- Order History: View past and current orders from the user profile
- Profile Management: Update personal information and delivery preferences

2.3.2 Restaurant Features

- Restaurant Authentication: Dedicated login for restaurant accounts

- Admin Approval Workflow: Restaurants must be approved by admin before listing items
- Product Listing Management: Add, edit, and remove food items from the menu
- Order Management: View and manage incoming orders via the Restaurant Dashboard

2.3.3 Admin Features

- Admin Dashboard: Centralized control panel for platform management
- User & Restaurant Management: View and manage all registered users and restaurants
- Order Oversight: Monitor all orders placed on the platform
- Category & Promotion Management: Configure promoted restaurants and food categories

3. Architecture

OrderOnTheGo follows the MERN stack architecture, implementing a three-tier application structure comprising a Presentation Layer (React.js), Application Layer (Node.js + Express.js), and Data Layer (MongoDB). This separation of concerns promotes modularity, scalability, and ease of maintenance.

3.1 Frontend Architecture

Technology: React.js with modern JavaScript (ES6+), Bootstrap for responsive styling.

The frontend is structured as a Single Page Application (SPA) using React.js. Components are organized modularly and communicate with the backend via RESTful API calls using the Fetch API or Axios. The key architectural elements include:

- Component-Based Design: Reusable UI components including Navbar, ProductCard, CartItem, AdminPanel, etc.
- React Router DOM: Client-side routing enabling navigation without full-page reloads
- State Management: React Hooks (useState, useEffect) for component-level state and Context API for global state such as authentication and cart data
- Axios / Fetch API: HTTP client for communicating with backend API endpoints
- Bootstrap: Responsive grid system and UI components for consistent cross-device rendering

3.2 Backend Architecture

Technology: Node.js runtime with Express.js framework.

The backend serves as the application layer, exposing a RESTful API consumed by the frontend. Key architectural decisions include:

- Express.js Server: Lightweight, unopinionated web framework handling HTTP routing and middleware
- MVC Pattern: Routes, Controllers, and Models are separated to promote clean code organization
- Middleware Stack: CORS for cross-origin requests, body-parser for JSON parsing, and custom authentication middleware for route protection
- Authentication: Session-based or token-based user verification applied to protected routes
- Error Handling: Centralized error handling middleware returning consistent HTTP error responses

3.3 Database Design and Schema

Technology: MongoDB (NoSQL Document Store), managed via Mongoose ODM.

MongoDB was selected for its flexible document-oriented schema, which aligns with the dynamic nature of food ordering data. The following collections are defined:

3.3.1 User Schema

Collection: users — Stores registered customer data including username, email address, hashed password, and delivery preferences.

3.3.2 Restaurant Schema

Collection: restaurants — Stores restaurant profile details, approval status from admin, location, and menu linkage.

3.3.3 Product Schema

Collection: products — Stores all food items available across restaurants, including item name, description, price, category, and associated restaurant ID.

3.3.4 Cart Schema

Collection: cart — Stores items added to cart by a specific user, referenced via userId. Includes product ID, quantity, and pricing snapshot.

3.3.5 Orders Schema

Collection: orders — Stores confirmed order records including userId, list of ordered products, delivery address, payment method, and order timestamp.

3.3.6 Admin Schema

Collection: admin — Stores administrative configurations including promoted restaurants and active food categories visible on the platform homepage.

4. Setup Instructions

4.1 Prerequisites

Before setting up the application locally, ensure that the following software dependencies are installed:

- Node.js (v16 or above) — JavaScript runtime for executing the backend server
- npm (v8 or above) — Node Package Manager, bundled with Node.js installation
- MongoDB — NoSQL database (local installation or cloud-based via MongoDB Atlas)
- Git — Version control system for cloning the repository
- Visual Studio Code (recommended) — Code editor for development

Download links for prerequisites:

- Node.js: https://nodejs.org/en/download/
- MongoDB: https://www.mongodb.com/try/download/community
- Git: https://git-scm.com/downloads
- Visual Studio Code: https://code.visualstudio.com/download

4.2 Installation Steps

Step 1 — Clone the Repository

Open a terminal or command prompt and execute the following:

```
git clone https://github.com/harsha-vardhan-reddy-07/Food-Ordering-App-MERN
cd Food-Ordering-App-MERN
```

Step 2 — Install Server Dependencies

Navigate to the server directory and install the required packages:

```
cd server
npm install
```

Step 3 — Install Client Dependencies

Navigate to the client directory and install the required packages:

```
cd ../client
npm install
```

Step 4 — Configure Environment Variables

Create a .env file in the server directory and populate it with the following configuration values:

```
MONGO_URI=<Your MongoDB Connection String>
PORT=5000
JWT_SECRET=<Your Secret Key>
```

For MongoDB Atlas, obtain your connection string from the Atlas dashboard under Database > Connect > Connect your application.

5. Folder Structure

5.1 Client (React Frontend)

The React application is organized under the /client directory following a component-based modular structure:

```
client/
├── public/                 # Static assets and index.html
├── src/
│   ├── components/         # Reusable UI components
│   │   ├── Navbar.js
│   │   ├── ProductCard.js
│   │   ├── CartItem.js
│   │   └── Footer.js
│   ├── pages/              # Full page-level components
│   │   ├── Home.js         # Landing page with restaurant listings
│   │   ├── Login.js
│   │   ├── Register.js
│   │   ├── Cart.js
│   │   ├── Profile.js
│   │   └── AdminDashboard.js
│   ├── context/            # React Context for global state
│   ├── App.js              # Root component with route definitions
│   └── index.js            # Application entry point
└── package.json
```

5.2 Server (Node.js Backend)

The Node.js backend is organized under the /server directory following an MVC-inspired pattern:

```
server/
├── models/                 # Mongoose schema definitions
│   ├── User.js
│   ├── Product.js
│   ├── Cart.js
│   ├── Orders.js
│   ├── Restaurant.js
│   └── Admin.js
├── routes/                 # Express route handlers
│   ├── userRoutes.js
```

```
│   ├── productRoutes.js
│   ├── cartRoutes.js
│   ├── orderRoutes.js
│   └── adminRoutes.js
├── middleware/          # Custom middleware (auth, error handling)
│   └── authMiddleware.js
├── config/              # Database connection configuration
│   └── db.js
├── index.js             # Express server entry point
└── package.json
```

6. Running the Application

6.1 Starting the Backend Server

Navigate to the server directory and execute the following command to start the Node.js Express server:

```
cd server
npm start
```

The backend server will start on http://localhost:5000 by default. The port can be modified via the PORT variable in the .env configuration file.

6.2 Starting the Frontend Server

In a separate terminal window, navigate to the client directory and execute:

```
cd client
npm start
```

The React development server will start on http://localhost:3000. The application will automatically open in the default web browser. Hot-reloading is enabled, so changes to source files are reflected immediately.

6.3 Accessing the Application

Once both servers are running simultaneously, the application can be accessed at the following URLs:

- User Interface: http://localhost:3000
- Backend API: http://localhost:5000/api
- Admin Dashboard: http://localhost:3000/admin (requires Admin credentials)
- Restaurant Dashboard: http://localhost:3000/restaurant (requires Restaurant credentials)

7. API Documentation

The backend exposes a RESTful API consumed by the React frontend. All API routes are prefixed with /api. JSON is the primary data exchange format for all request and response bodies.

7.1 User Authentication Endpoints

Method	Endpoint	Description	Body / Params	Response
POST	/api/users/register	Register a new user	{ name, email, password }	201: { message, userId }
POST	/api/users/login	Authenticate user and return token	{ email, password }	200: { token, user }
GET	/api/users/profile	Fetch authenticated user profile	Bearer Token (Header)	200: { user object }
PUT	/api/users/profile	Update user profile details	{ name, address }	200: { updated user }

7.2 Product Endpoints

Method	Endpoint	Description	Body / Params	Response
GET	/api/products	Retrieve all available food items	None	200: [array of products]
GET	/api/products/:id	Retrieve a specific product by ID	Product ID in URL	200: { product object }
POST	/api/products	Add a new food item (Admin/Restaurant)	{ name, price, description, category, restaurantId }	201: { created product }
PUT	/api/products/:id	Update a product (Admin/Restaurant)	{ name, price, description }	200: { updated product }
DELETE	/api/products/:id	Delete a product (Admin/Restaurant)	Product ID in URL	200: { message }

7.3 Cart Endpoints

Method	Endpoint	Description	Body / Params	Response
GET	/api/cart/:userId	Retrieve user's current cart	User ID in URL	200: [cart items]

Method	Endpoint	Description	Body / Params	Response
POST	/api/cart	Add an item to the cart	{ userId, productId, quantity }	201: { cart item }
PUT	/api/cart/:itemId	Update quantity of a cart item	{ quantity }	200: { updated item }
DELETE	/api/cart/:itemId	Remove an item from the cart	Item ID in URL	200: { message }

7.4 Order Endpoints

Method	Endpoint	Description	Body / Params	Response
POST	/api/orders	Place a new order from cart	{ userId, items, address, paymentMethod }	201: { order confirmation }
GET	/api/orders/user/:userId	Retrieve all orders of a user	User ID in URL	200: [order list]
GET	/api/orders	Retrieve all platform orders (Admin)	Admin Bearer Token	200: [all orders]
PUT	/api/orders/:orderId	Update order status (Admin)	{ status }	200: { updated order }

7.5 Admin Endpoints

Method	Endpoint	Description	Body / Params	Response
GET	/api/admin/users	Retrieve all registered users	Admin Token	200: [users list]
GET	/api/admin/restaurants	Retrieve all restaurants	Admin Token	200: [restaurants list]
PUT	/api/admin/restaurants/:id/approve	Approve a restaurant listing	Admin Token	200: { message }
GET	/api/admin/orders	Retrieve all orders on the platform	Admin Token	200: [orders]

8. Authentication

8.1 Overview

OrderOnTheGo implements a role-based authentication and authorization system to ensure that only authorized users can access specific platform features. The application supports three distinct user roles: Customer, Restaurant Owner, and Administrator.

8.2 Authentication Mechanism

User authentication is handled through a combination of password hashing and JSON Web Tokens (JWT):

8.2.1 Password Security

User passwords are never stored in plaintext. During registration, the plaintext password is hashed using bcrypt, a cryptographic hashing algorithm with an adaptive cost factor. The resulting hash is stored in the MongoDB users collection. During login, the submitted password is compared against the stored hash using bcrypt.compare().

8.2.2 JSON Web Tokens (JWT)

Upon successful login, the server generates a signed JWT containing the user's ID and role. This token is returned to the client and stored (typically in localStorage or a cookie). For all subsequent requests to protected routes, the client includes the token in the Authorization header using the Bearer scheme.

```
Authorization: Bearer <token>
```

The token is verified on the server side using custom authentication middleware before any protected route handler is executed. If the token is invalid, expired, or absent, the server returns a 401 Unauthorized response.

8.3 Authorization and Role-Based Access Control

Authorization is enforced through middleware that inspects the role field embedded in the JWT payload. The three roles and their associated access levels are defined as follows:

- Customer: Access to product browsing, cart management, order placement, and personal profile.
- Restaurant Owner: Access to restaurant dashboard, product listing management, and restaurant-specific order management. Requires admin approval before elevated access is granted.

- Administrator: Unrestricted access to all platform resources including user management, restaurant approval, and order oversight via the Admin Dashboard.

8.4 Application Flow Summary

The authentication flow proceeds as follows: a user submits credentials via the login form; the server validates credentials, generates a JWT, and returns it; the React frontend stores the token and attaches it to subsequent API requests; the server middleware verifies the token and extracts role information; access is granted or denied based on the user's role.

9. User Interface

9.1 Overview

The OrderOnTheGo user interface is designed with simplicity and accessibility in mind. The application adopts a clean, modern aesthetic built using Bootstrap and custom CSS, ensuring a consistent and responsive experience across desktop and mobile devices. The UI is organized around four primary user journeys: Customer Ordering, Restaurant Management, Admin Control, and User Authentication.

9.2 Key UI Screens

9.2.1 Landing Page

The landing page presents users with a visually engaging homepage displaying featured restaurants and food categories. It serves as the primary entry point and encourages exploration through prominent call-to-action elements.

9.2.2 Restaurant Listing Page

This screen presents all available restaurants on the platform with filtering options. Users can browse by category, view restaurant ratings, and navigate to individual restaurant menus.

9.2.3 Restaurant Menu Page

Upon selecting a restaurant, users are presented with the full menu. Each food item displays its name, description, price, and an 'Add to Cart' button. Items can be added to the cart directly from this screen.

9.2.4 Authentication Screens

The Login and Registration screens provide clean form-based interfaces for user account management. Input validation and error feedback are displayed inline to guide users through the process.

9.2.5 Shopping Cart

The Cart screen displays all items currently added by the user. Users can adjust quantities, remove items, and review the total amount before proceeding to checkout.

9.2.6 User Profile

The Profile screen allows authenticated users to view and update their personal information, delivery address, and order history.

9.2.7 Admin Dashboard

The Admin Dashboard provides a comprehensive control panel with tabbed sections for managing users, restaurants, orders, and platform categories. Data is presented in tabular format for easy review.

9.2.8 Restaurant Dashboard

Restaurant owners access a dedicated dashboard to add and manage food items, view incoming orders, and track their restaurant's performance on the platform.

Note: Screenshots of all UI screens are included in Section 11 (Screenshots and Demo) of this document.

10. Testing

10.1 Testing Strategy

The OrderOnTheGo application was tested using a combination of manual testing and unit testing approaches to validate functional correctness, API reliability, and UI behavior. Testing was conducted iteratively throughout the development lifecycle to identify and resolve issues at the earliest opportunity.

10.2 Manual Testing

Manual testing was conducted across all primary user flows to verify end-to-end functionality. The test scenarios covered the following areas:

- User Registration and Login: Verified successful registration, login with valid credentials, and rejection of invalid credentials.
- Product Browsing and Filtering: Confirmed that all products load correctly, filters apply accurately, and search results match expected outcomes.
- Cart Operations: Tested adding items to cart, updating quantities, and removing items. Verified that the cart total updates dynamically.
- Order Placement: Validated the full checkout flow from cart to order confirmation, including address input and payment method selection.
- Admin Dashboard: Verified admin access control, user management, restaurant approval, and order overview functionalities.
- Restaurant Dashboard: Tested product addition, editing, and deletion by a restaurant owner account.
- Authentication: Confirmed JWT token generation, protected route enforcement, and session expiration behavior.

10.3 API Testing

Backend API endpoints were tested using Postman, a widely used API testing tool. Each endpoint was tested for correct HTTP status codes, expected response bodies, error handling, and authorization enforcement. Test cases included valid requests, invalid payloads, missing authentication tokens, and boundary conditions.

10.4 Browser Compatibility

The application UI was tested across the following browsers to ensure consistent rendering and functionality: Google Chrome (latest), Mozilla Firefox (latest), and Microsoft Edge (latest). No major cross-browser compatibility issues were identified.

10.5 Responsive Design Testing

The responsive layout was validated across multiple viewport sizes including desktop (1920px), laptop (1366px), tablet (768px), and mobile (375px) using Chrome DevTools. Bootstrap's grid system ensured correct reflow of UI elements across all tested screen sizes.

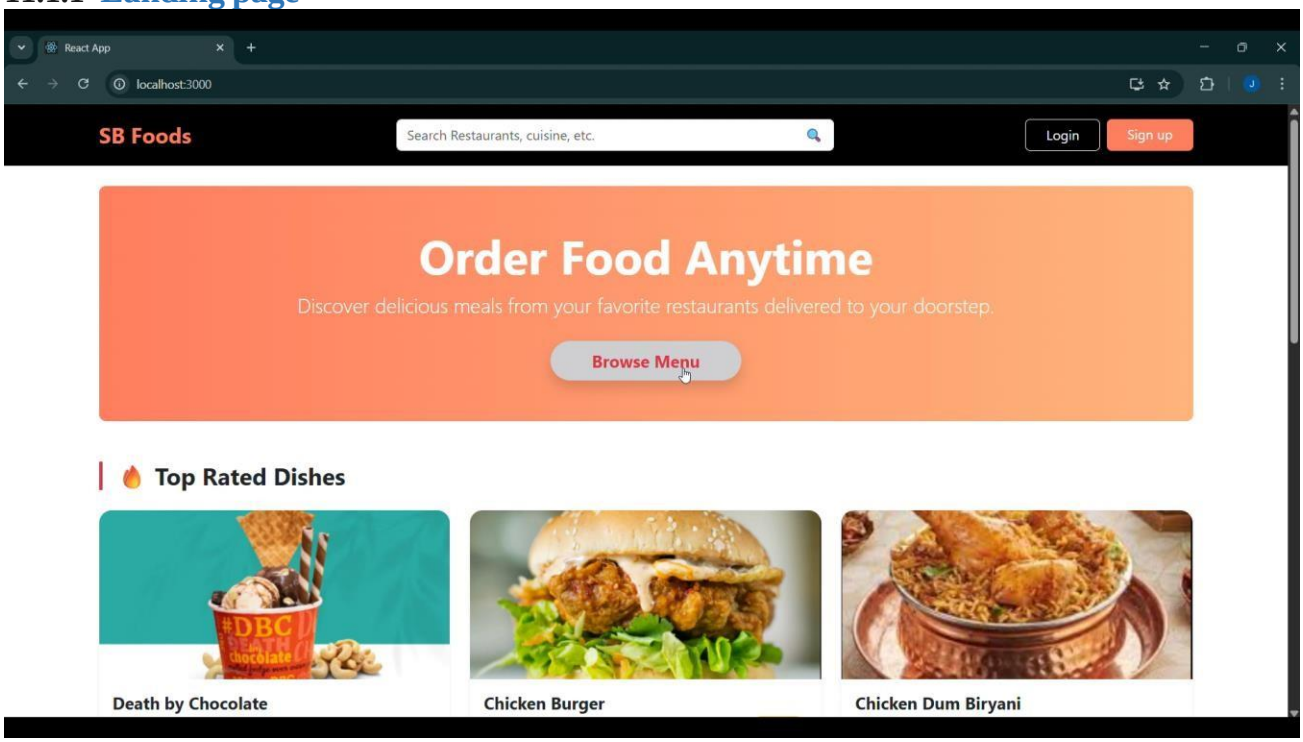
11. Screenshots and Demo

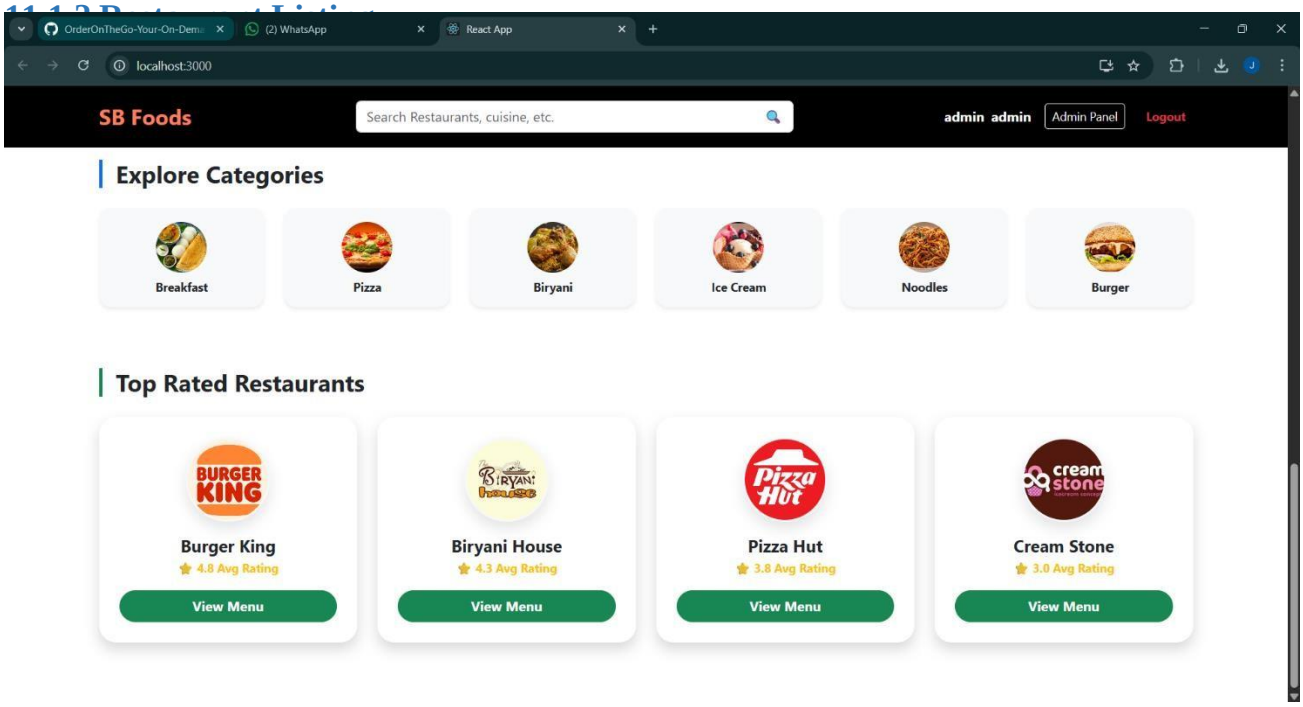
11.1 Application Screenshots

The following table summarizes the key UI screens implemented in the OrderOnTheGo application. Screenshots demonstrating each screen are available in the project submission package and the demo video linked below.

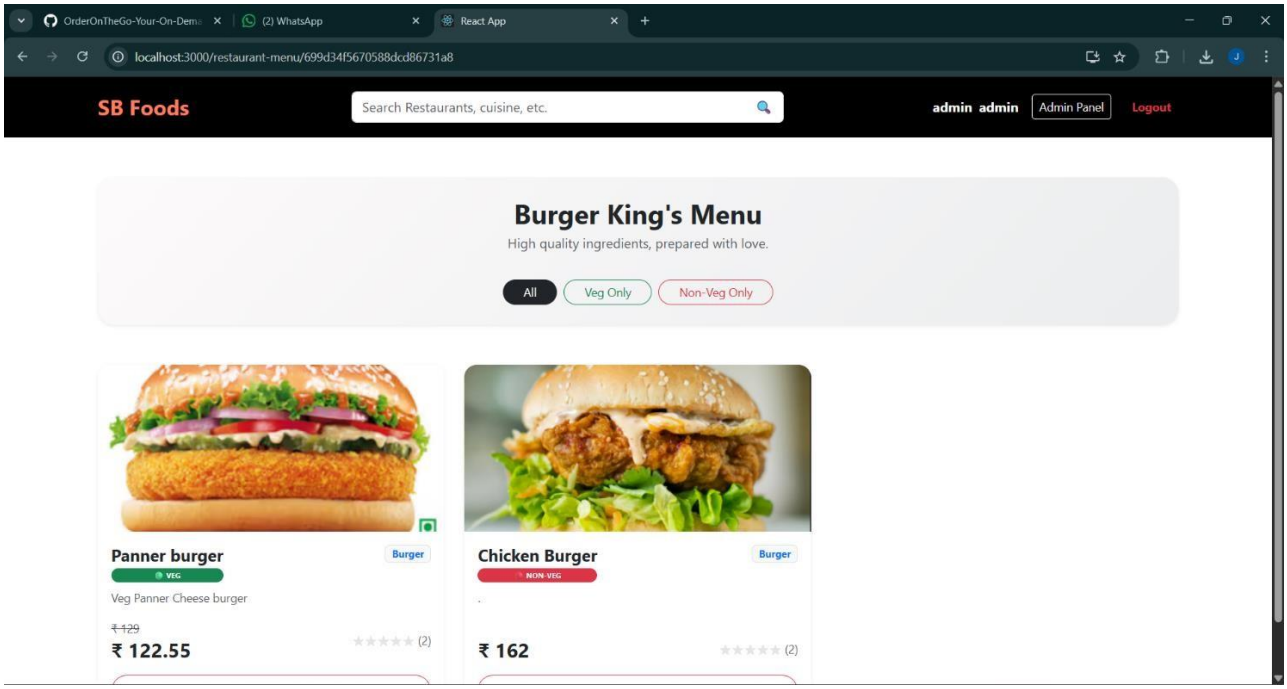
Screen	Description
Landing Page	Entry point displaying featured restaurants and categories
Restaurant Listing	Grid view of all available restaurants with filters
Restaurant Menu	Food items for a selected restaurant with Add to Cart
User Login / Register	Authentication forms with input validation
Shopping Cart	Cart summary with quantity controls and total amount
User Profile	Personal info, delivery address, and order history
Admin Dashboard	Platform management panel for users, restaurants, orders
Restaurant Dashboard	Restaurant owner panel for menu and order management
New Product Form	Admin/Restaurant form to add new food items

11.1.1 Landing page

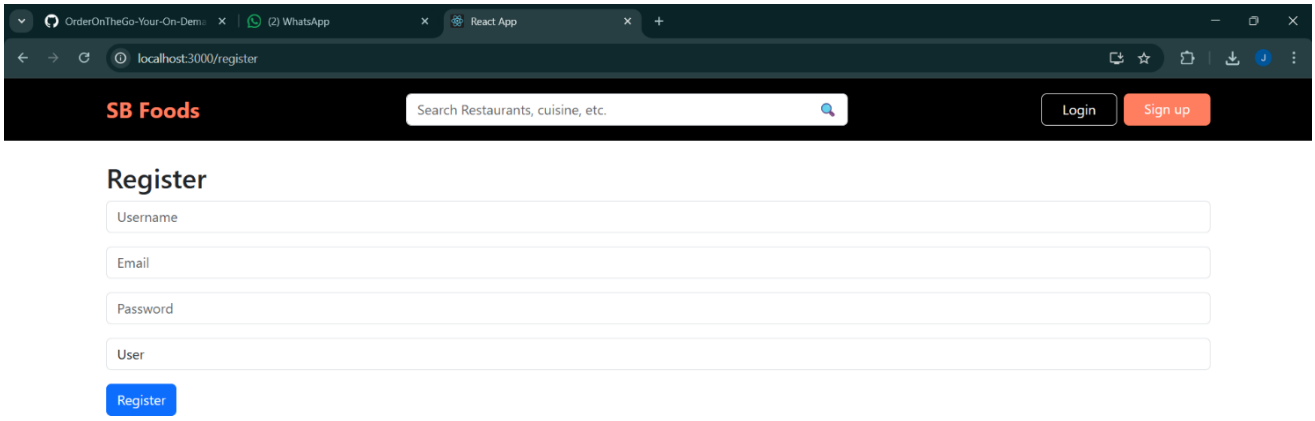




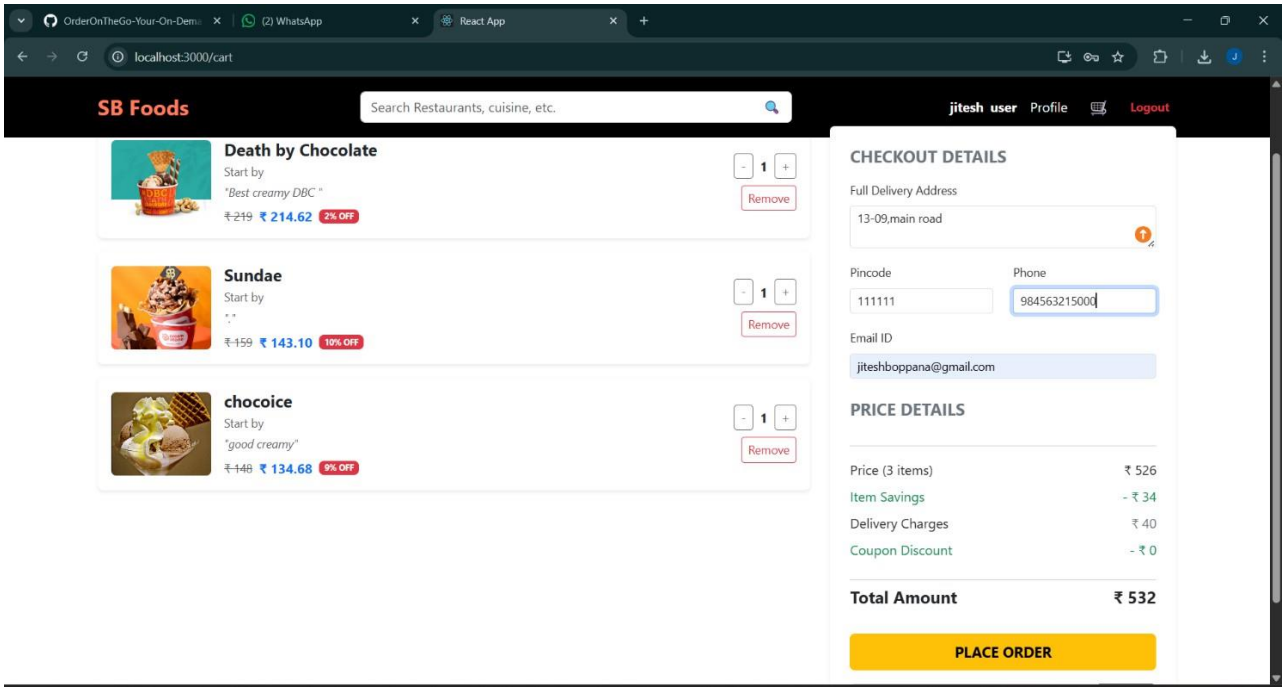
11.1.3 Restaurant menu



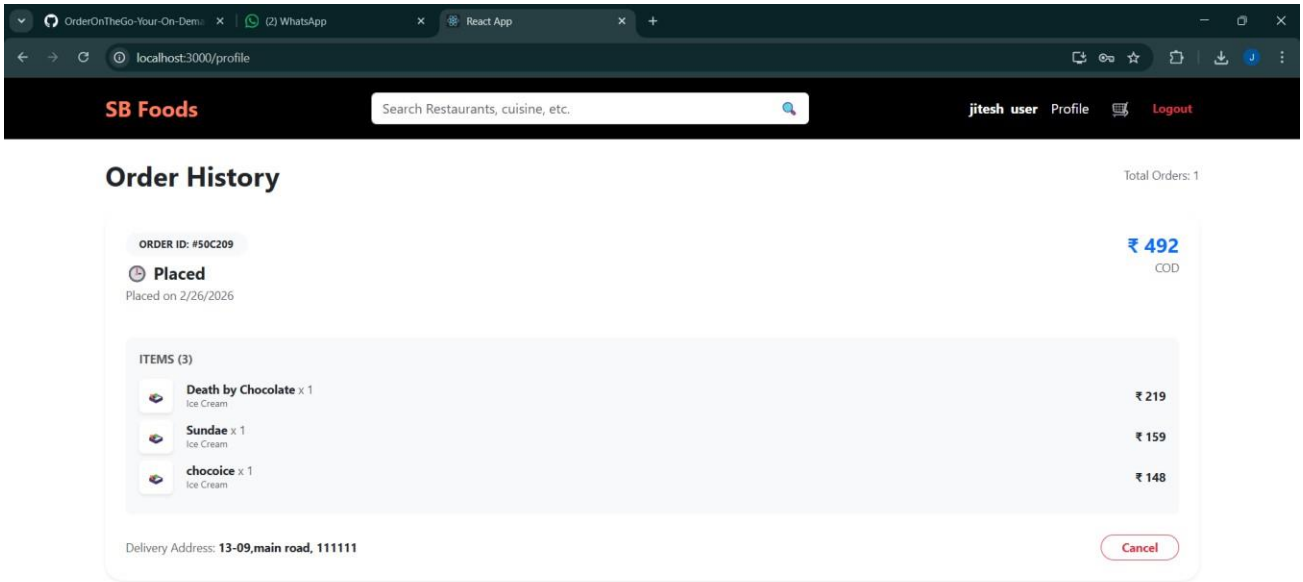
11.1.4 User Login/Register



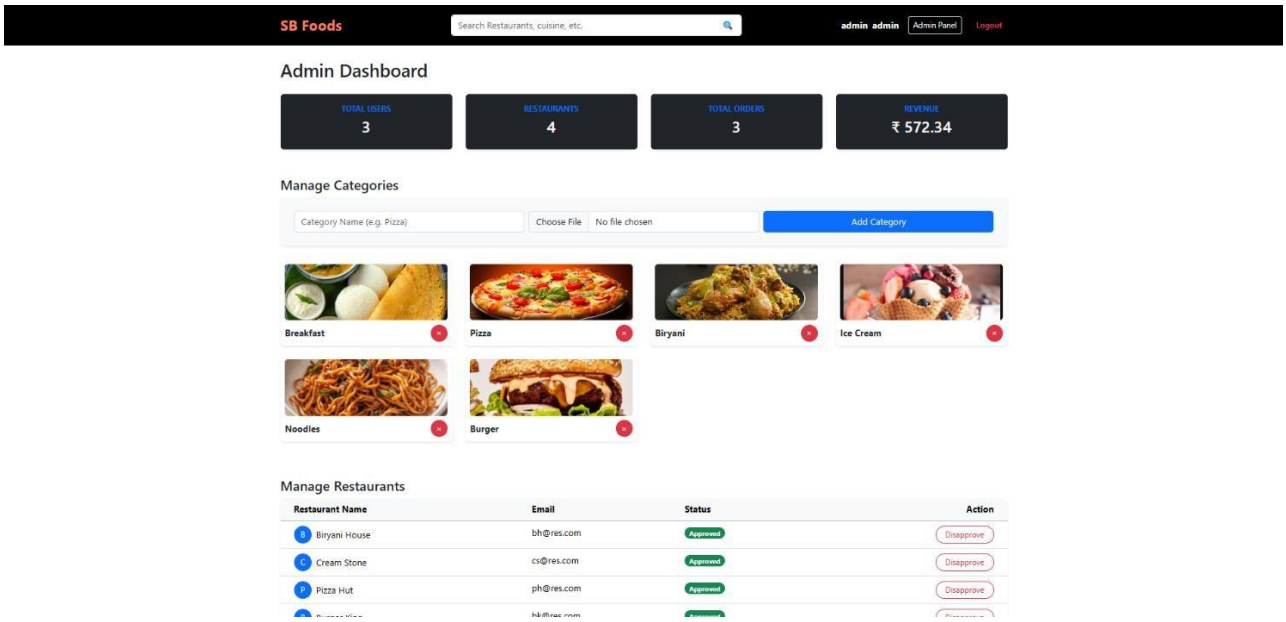
11.1.5 Shopping Cart



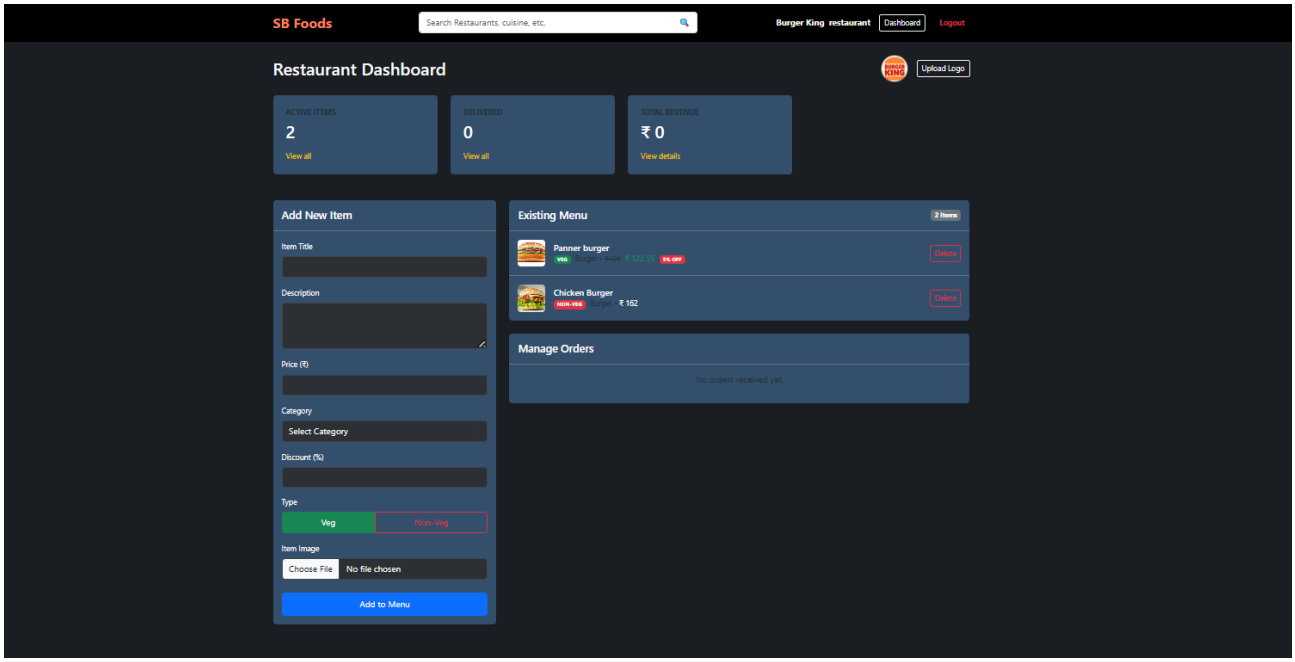
11.1.6 User Profile



11.1.7 Admin Dashboard



11.1.8 Restaurant Dashboard and New Product Form



11.2 Demo Video

A recorded demonstration of the fully functional OrderOnTheGo application is available at the following link:

DemoVideo:
https://drive.google.com/file/d/1utoE8BK08k6MqRY66O6NAyMLU1F4jdkG/view?usp=drive_link

The demo video walks through all major user flows including user registration, restaurant browsing, cart management, order placement, and admin dashboard operations.

12. Known Issues

During the development and testing phases, the following known issues were identified. These are documented for transparency and will be addressed in future iterations of the project.

- Payment Gateway Integration: The current implementation uses a simulated payment method selection rather than integration with a live payment gateway (e.g., Stripe or Razorpay). Orders are confirmed without actual payment processing.
- Real-Time Order Tracking: The application does not currently support real-time order status updates. Users must manually refresh the orders page to view status changes.
- Image Upload for Products: Product images are currently managed via static URLs rather than a direct file upload mechanism. A dedicated image hosting solution (e.g., Cloudinary) is planned.
- Session Persistence: In certain browsers, the JWT stored in localStorage may be cleared upon closing the browser window, requiring users to log in again.
- Input Validation: Some form fields on the frontend lack comprehensive client-side validation, which may result in suboptimal user experience for edge-case inputs.
- Mobile Navigation: On smaller screen sizes, the navigation menu requires minor CSS refinements to improve usability.
- Restaurant Search: A dedicated search-by-restaurant-name feature is not yet implemented; current filtering is category-based only.

13. Future Enhancements

The following enhancements have been identified as high-priority improvements for future development cycles of the OrderOnTheGo platform:

13.1 Payment Gateway Integration

Integration with a production-grade payment gateway such as Razorpay or Stripe will be implemented to enable secure real-money transactions. This will include support for multiple payment modes such as UPI, credit/debit cards, and net banking.

13.2 Real-Time Order Tracking

WebSocket integration (e.g., using Socket.io) will be introduced to provide real-time order status updates to customers. Users will receive live notifications at each stage of the delivery lifecycle.

13.3 Mobile Application

A native mobile application for Android and iOS platforms will be developed using React Native, extending the platform's reach to smartphone users with an optimized mobile experience.

13.4 Advanced Search and Recommendation Engine

A machine learning-powered recommendation engine will be integrated to suggest personalized food items and restaurants based on a user's order history and browsing behavior.

13.5 Customer Reviews and Ratings

A comprehensive review and rating system will be implemented, allowing users to rate food items and restaurants after order delivery, fostering transparency and community engagement.

13.6 Delivery Partner Module

An additional user role — Delivery Partner — will be introduced with a dedicated interface for accepting delivery assignments, updating order statuses in real time, and navigating to delivery locations.

13.7 Analytics Dashboard

An enhanced admin analytics dashboard will be developed to provide visual insights into platform performance including total orders, revenue trends, top-selling items, and active user metrics.

13.8 Multilingual Support

Internationalization (i18n) will be implemented using libraries such as i18next to support multiple regional languages, broadening the platform's accessibility to diverse user demographics.

14. Conclusion

OrderOnTheGo (SB Foods) represents a fully functional, end-to-end food ordering platform developed using the MERN stack as part of the SmartBridge Full Stack Development program. The project demonstrates the practical application of modern web development technologies including React.js, Node.js, Express.js, and MongoDB to solve a real-world problem — delivering a seamless, digital food ordering experience.

Through the development of this application, the team gained hands-on experience in full-stack architecture design, RESTful API development, component-based UI development, database schema design, and multi-role authentication. The modular and scalable architecture ensures that the platform can evolve with future feature additions and growing user demands.

The known issues documented in this report serve as a transparent roadmap for improvement, and the future enhancements outlined provide a clear direction for the platform's long-term growth. OrderOnTheGo is well-positioned to serve as a foundation for a production-grade food delivery service.

15. References

1. React.js Official Documentation — https://reactjs.org/docs/
2. Node.js Official Documentation — https://nodejs.org/en/docs/
3. Express.js Official Documentation — https://expressjs.com/
4. MongoDB Official Documentation — https://www.mongodb.com/docs/
5. Mongoose ODM Documentation — https://mongoosejs.com/docs/
6. Bootstrap Documentation — https://getbootstrap.com/docs/
7. JSON Web Tokens (JWT) — https://jwt.io/introduction/
8. bcrypt npm package — https://www.npmjs.com/package/bcrypt
9. Git Installation Guide — https://git-scm.com/book/en/v2/Getting-Started-Installing-Git
10. MongoDB Atlas Setup — https://www.mongodb.com/docs/atlas/tutorial/connect-to-your-cluster/
11. Git Documentation. (2024). Git - Version Control System. https://git-scm.com/doc

Acknowledgments

I would like to express our sincere gratitude to all those who contributed to the successful completion of the OrderOnTheGo project.

Special thanks to Ganesh Maddi for their guidance, support, and valuable insights throughout the project development. Their expertise in full-stack development and constructive feedback were instrumental in shaping this project.

We are grateful to SmartBridge, SmartInternz, APSCHE for providing the resources and environment conducive to learning and innovation. The knowledge and skills acquired during our coursework formed the foundation for this project.

We also acknowledge the open-source community for the excellent tools and frameworks that made this project possible, including React.js, Node.js, Express.js, and MongoDB.

Finally, we thank our families and peers for their continuous encouragement and support throughout this journey.